

□ Bitácora de Prompts — Huerto Vivo

Documento que evidencia los prompts más efectivos utilizados en **Claude (Anthropic)** para construir el sistema de gestión del huerto comunitario urbano.

Proyecto: Huerto Vivo — Sistema de gestión comunitaria

Herramienta: Claude (Anthropic) · claude.ai

Stack: PostgreSQL · Node.js / Express / TypeScript · React / Vite **Celso Cereceda - Hernan Cortes - Stefano Morales - Sofia Riveros

Alcance: Base de Datos · Backend · Frontend

Índice

- [Fase 1 — Base de Datos](#)
 - [Fase 2 — Backend](#)
 - [Fase 3 — Frontend](#)
 - [Patrones de Efectividad](#)
-

Fase 1 — Base de Datos

Objetivo: Diseñar el esquema relacional que resuelva cada pain point del proyecto a nivel estructural, garantizando integridad de datos y trazabilidad completa de las acciones de los 18 vecinos.

Prompt #1 — Generación del esquema relacional completo

Contexto:

Con los pain points, requerimientos y stack tecnológico ya definidos en la conversación, se solicitó la base de datos completa en un único prompt. Claude tenía el dominio del problema, las entidades lógicas identificadas en el análisis y el objetivo del sistema.

Prompt utilizado:

quiero que me ayudes a generar una base de datos DDL Y DML con postgresQL, tomando en cuenta todo el contexto del proyecto: 18 vecinos gestionando un huerto comunitario, con necesidad de registrar siembras (qué, dónde, cuándo, quién), turnos de riego diarios con responsable asignado, cosechas con distribución equitativa, y un sistema de métricas de actividad por vecino. La base debe incluir tipos ENUM para estados, constraints que eviten duplicados, índices de optimización y datos de prueba completos.

Por qué fue efectivo:

La efectividad radica en la precisión del dominio entregado. El prompt no pidió "una base de datos" genérica, sino que especificó las 4 entidades principales del negocio (siembras, riegos, cosechas, métricas), las reglas de integridad esperadas (constraints, ENUMs) y los requerimientos de datos de

prueba. Esto permitió que Claude infiriera las relaciones entre tablas y los tipos de datos apropiados sin necesidad de iteraciones adicionales.

Resultado obtenido:

- **7 tablas relacionales** con claves foráneas correctas y sin redundancias: `vecinos`, `parcelas`, `especies`, `siembras`, `turnos_riego`, `cosechas`, `distribuciones_cosecha`
 - **4 tipos ENUM** para garantizar integridad: `estado_turno`, `estado_parcela`, `rol_vecino`, `estado_cosecha`
 - **Constraint único** `uq_parcela_activa` que impide físicamente dos siembras activas en la misma parcela — resuelve el pain point de duplicación espacial a nivel de base de datos
 - **3 vistas SQL** que responden directamente las preguntas del dashboard: `v_riego_hoy`, `v_cosechas_mes`, `v_ranking_vecinos`
 - **DML completo** con 18 vecinos, 18 parcelas en grilla 6×3, 8 especies y datos de turnos de los últimos 7 días
-

Prompt #2 — Refactorización total con autenticación integrada

Contexto:

Después de construir el backend, se detectó que la base de datos necesitaba soporte de autenticación real (login con email y contraseña), un sistema de chat persistente y notificaciones tipadas. En lugar de agregar parches, se optó por una refactorización completa.

Prompt utilizado:

```
quiero que empecemos desde la base de datos, haciéndola nuevamente con todo lo necesario, desde el inicio de sesión, hasta todo lo que llevamos. Necesito que incluya: autenticación con password_hash usando bcrypt para los 18 vecinos, tabla de mensajes_chat para reemplazar el grupo de WhatsApp, tabla de notificaciones con tipos (info, riego, cosecha, alerta), y tabla de actividad_vecinos para el ranking mensual de compromiso. Los hashes de contraseña deben ser bcrypt válidos de la clave '1234' para facilitar las pruebas. Incluir script de verificación al final con conteo de registros.
```

Por qué fue efectivo:

Solicitar una refactorización total en el momento preciso — cuando el proyecto había madurado — fue más eficiente que hacer cambios incrementales. La frase "*desde el inicio de sesión*" actuó como ancla semántica que garantizó que la autenticación quedara integrada en el diseño desde el principio, no agregada como parche. Especificar que los hashes debían ser bcrypt válidos de '1234' fue un detalle técnico crítico que ahorró una iteración completa de configuración.

Resultado obtenido:

- **Versión 2.0 con 10 tablas:** se agregaron `mensajes_chat`, `notificaciones` y `actividad_vecinos`
- **Columna password_hash** en `vecinos` con hashes bcrypt pregenerados — autenticación lista para usar sin configuración adicional

- **Tabla actividad_vecinos** con métricas mensuales (turnos asignados, cumplidos, incumplidos, kg cosechados) — base del ranking de compromiso
 - **Índices de optimización** en columnas de búsqueda frecuente: fecha_turno, vecino_id, created_at
 - **Script de verificación** al final con `SELECT COUNT( *)` de cada tabla para confirmar inserción correcta
-

Prompt #3 — Corrección de error de tipos ENUM

Contexto:

Al ejecutar el script en pgAdmin apareció un error de tipo en el INSERT de turnos_riego. Se reportó con captura de pantalla del error exacto en lugar de describirlo en texto.

Prompt utilizado:

[imagen del error en pgAdmin]

```
"Failed to run sql query: ERROR: 42804: column "estado" is of type estado_turno but expression is of type text – LINE 353"
```

me dio este error, cómo lo soluciono sin romper el resto del script?

Por qué fue efectivo:

Reportar el error con captura de pantalla fue determinante. La imagen mostró la línea exacta (353), el nombre de la columna, el tipo esperado y el código de error PostgreSQL (42804) — información que habría tomado varios intercambios obtener por texto. Agregar *"sin romper el resto del script"* previno una solución invasiva y garantizó un fix quirúrgico.

Resultado obtenido:

- **Causa identificada:** PostgreSQL no convierte automáticamente texto a ENUM en bloques VALUES — requiere cast explícito
 - **Fix aplicado:** agregar `::estado_turno` en el INSERT — solución de una línea
 - **Fix preventivo:** se revisaron y corrigieron todos los ENUMs del script (`::estado_parcela`, `::rol_vecino`, `::estado_cosecha`) antes de que causaran el mismo error
 - **Tiempo de resolución:** un solo intercambio sin iteraciones adicionales
-

Fase 2 — Backend

Objetivo: Construir la API REST conectada a la base de datos, con autenticación JWT, validación de datos y rutas organizadas por módulo funcional.

Prompt #4 — Servidor REST completo en un prompt

Contexto:

Con la base de datos operativa y el stack definido (Node.js + Express + TypeScript + Zod + JWT), se solicitó el backend completo especificando la arquitectura esperada.

Prompt utilizado:

```
ahora necesito el backend completo en Node.js + Express + TypeScript,
conectado a la base de datos PostgreSQL que ya tenemos. Debe incluir:
- Autenticación JWT con bcrypt (login contra tabla vecinos)
- Control de roles: coordinador y miembro con middleware soloCoordinador
- Validación de esquemas con Zod en todos los endpoints POST y PATCH
- Transacciones en operaciones críticas (registrar siembra bloquea parcela,
reportar cosecha actualiza métricas de actividad)
- Rutas para: auth, vecinos, parcelas, siembras, turnos de riego, cosechas,
dashboard, chat y notificaciones
- Health check en GET /health que prueba la conexión real a PostgreSQL
- Conexión al pool con detección automática de entorno (local vs cloud)
```

Por qué fue efectivo:

La estructura del prompt reflejó la arquitectura real del backend antes de pedirlo. Al listar explícitamente las transacciones críticas, el control de roles y la validación como requerimientos separados, Claude pudo generar cada capa de forma independiente y coherente. Especificar *"detección automática de entorno"* para la conexión SSL anticipó el problema más común al mover el proyecto entre desarrollo local y producción cloud.

Resultado obtenido:

- **9 módulos de rutas** independientes: auth, vecinos, parcelas, siembras, turnos, cosechas, dashboard, chat, notificaciones
- **Middleware JWT** con authenticate y soloCoordinador — bloquea rutas sensibles para miembros regulares
- **Validación Zod** en todos los endpoints POST y PATCH — rechaza datos malformados antes de tocar la BD
- **Transacciones atómicas:** registrar siembra bloquea la parcela y reportar cosecha actualiza actividad_vecinos en la misma transacción
- **Health check real** en GET /health que ejecuta SELECT 1 contra PostgreSQL antes de responder

Prompt #5 — Diagnóstico y corrección de error SSL

Contexto:

Al iniciar el backend, el health check devolvía db_error. Se modificó el endpoint para mostrar el detalle del error y se reportó con captura de pantalla del navegador.

Prompt utilizado:

```
[imagen del navegador mostrando]
{"ok":false,"status":"db_error","detalle":"Error: The server does not
support SSL connections"}
```

modifica la conexión en db.ts para que funcione tanto en PostgreSQL local (sin SSL) como en Supabase o servicios cloud (con SSL), sin necesitar cambiar el archivo .env entre entornos

Por qué fue efectivo:

La combinación de la imagen del error con la restricción *"sin necesitar cambiar el .env"* fue la clave. Sin esa restricción, la solución más simple habría sido agregar `ssl: false` hardcodeado — funcional pero frágil al cambiar de entorno. La restricción forzó una solución robusta que detecta el entorno automáticamente sin configuración adicional.

Resultado obtenido:

- **Causa identificada:** `ssl: { rejectUnauthorized: false }` funciona en Supabase pero falla en PostgreSQL local
- **Fix elegante:** `const isLocal = DATABASE_URL.includes('localhost') || DATABASE_URL.includes('127.0.0.1')`
- **Resultado:** `ssl: false` para PostgreSQL local, `ssl: { rejectUnauthorized: false }` para servicios cloud
- **Beneficio adicional:** el mismo código funciona en desarrollo y producción sin tocar el .env

Prompt #6 — Rutas avanzadas de rotación automática y mapa

Contexto:

Con el backend base funcionando, se necesitaron módulos adicionales para las funcionalidades exclusivas del coordinador: rotación automática de turnos del mes entero y mapa visual del huerto con estado de riego en tiempo real.

Prompt utilizado:

necesito agregar dos módulos nuevos al backend:

1. /api/rotacion con estas rutas:
 - POST /generar: recibe fecha_inicio, fecha_fin y dias_intervalo. Distribuye turnos entre todos los vecinos activos rotando entre las parcelas con siembras activas. Al crear cada turno genera automáticamente una notificación para el vecino responsable. Usar transacción para toda la operación.
 - GET /preview: misma lógica pero sin guardar en BD, devuelve los primeros 30 turnos como vista previa para revisar antes de confirmar
 - POST /recordatorio: crea notificaciones para todos los vecinos con turno pendiente mañana
 - DELETE /limpiar: elimina turnos pendientes futuros desde una fecha dada para poder regenerar
2. /api/mapa:
 - GET /: devuelve todas las parcelas con cultivo activo, progreso de crecimiento calculado, estado de riego de hoy y responsable. Organizadas por coordenadas (ubicacion_fila, ubicacion_col).

Solo coordinador puede acceder a rotacion. Mapa accesible para todos.

Por qué fue efectivo:

Describir cada ruta con su comportamiento desde la perspectiva del usuario — no desde la perspectiva técnica — fue la decisión clave. *"Distribuye turnos entre todos los vecinos activos rotando entre las parcelas"* es una descripción de negocio que Claude tradujo correctamente a lógica de programación, JOINS necesarios y transacciones masivas. Especificar el endpoint de preview como *"misma lógica pero sin guardar"* definió la arquitectura del módulo de forma implícita.

Resultado obtenido:

- **POST /api/rotacion/generar:** genera turnos del mes con lógica de rotación configurable (intervalo 1-7 días) y crea notificaciones automáticas por vecino en una sola transacción
 - **GET /api/rotacion/preview:** misma lógica sin persistir — devuelve array para revisar antes de confirmar
 - **POST /api/rotacion/recordatorio:** busca todos los turnos pendientes de mañana y crea notificaciones individuales
 - **GET /api/mapa:** grilla completa con progreso calculado con `LEAST(100, ROUND( . . . ) )` y estado de riego del día actual
-

Fase 3 — Frontend

Objetivo: Construir la interfaz mobile-first conectada al backend, con navegación por roles, modo offline y experiencia visual inspirada en el prototipo de Google Stitch.

Prompt #7 — App completa desde imagen de prototipo

Contexto:

Con backend y base de datos operativos, se solicitó el frontend usando como referencia una captura del prototipo creado en Google Stitch mostrando 5 pantallas: Dashboard, Siembras, Riego, Cosecha y Chat.

Prompt utilizado:

[imagen del prototipo Stitch con 5 pantallas: Dashboard de Inicio, Registro de Siembra, Riego Interactivo, Cosecha y Chat Comunitario]

ya tengo listo la base de datos y el backend, ayúdame a generar el frontend en React + Vite inspirado en este prototipo. Debe ser mobile-first con navegación inferior de 5 tabs. Conectar cada pantalla a los endpoints reales del backend usando un api.js centralizado con todas las funciones. Incluir modo demo automático si el backend no responde para que la app sea navegable sin infraestructura activa.

Por qué fue efectivo:

Este fue el prompt más poderoso del proyecto por la combinación de tres elementos: el contexto

técnico acumulado ("*ya tengo listo el backend*"), la referencia visual concreta (imagen del prototipo) y el requerimiento de modo demo. La imagen aportó la paleta de colores, la estructura de navegación y el estilo visual sin necesidad de especificarlos en texto. La palabra "*inspirado*" fue clave — indicó adaptación creativa, no copia exacta.

Resultado obtenido:

- **5 pantallas funcionales:** Inicio (dashboard con KPIs), Siembras, Riego, Cosecha y Chat — cada una conectada a sus endpoints correspondientes
- **api.js centralizado** con 22 funciones que cubren todos los endpoints del backend
- **Paleta verde comunitaria** extraída automáticamente del prototipo con variables CSS
- **Navegación inferior** con tabs, modales con animación `slideUp`, tarjetas con sombras y bordes redondeados
- **Modo demo automático:** si el backend no responde, la app usa datos mock y sigue siendo funcional para demostración

Prompt #8 — Integración real con backend y funciones clave

Contexto:

El frontend inicial tenía las pantallas construidas pero las acciones principales no persistían en la base de datos. Se necesitaba conectar cada interacción al backend real con datos reales.

Prompt utilizado:

quiero que el frontend quede completamente entrelazado con el backend y la base de datos. Específicamente necesito:

- Activar campana de notificaciones con badge de no leídas y panel deslizable que muestre las notificaciones tipadas por color
- Agregar formulario desglosado para reportar cosecha: selector de siembra activa (desde BD), calidad percibida con botones visuales, cantidad en kg, fecha y notas opcionales
- Arreglar el orden del ranking de vecinos para que use `pct_cumplimiento` de la base de datos ordenado de mayor a menor
- Cuando el vecino marque su turno de riego, que el botón haga `PATCH` a `/api/turnos/:id/cumplir` y guarde en la BD con timestamp exacto
- El chat debe enviar mensajes reales con `POST /api/chat` y refrescar automáticamente cada 8 segundos con `GET /api/chat`

Por qué fue efectivo:

La estructura de lista con verbos imperativos específicos fue determinante. Cada ítem tenía un verbo de acción claro: *activar*, *agregar*, *arreglar*, *guarde*, *enviar*. Esta estructura permitió que Claude tratara cada requerimiento de forma independiente y verificara que cada uno quedara implementado. A diferencia de una descripción en prosa, la lista impidió que algún requerimiento quedara sin abordar.

Resultado obtenido:

- **`PATCH /api/turnos/:id/cumplir`** conectado al botón principal — cada click guarda en PostgreSQL con `hora_cumplido = NOW()`

- **Chat persistente:** POST /api/chat guarda cada mensaje, GET /api/chat recupera historial, autorefresh con setInterval de 8 segundos
 - **Ranking ordenado** por pct_cumplimiento DESC calculado en la vista v_ranking_vecinos de PostgreSQL — no en el frontend
 - **Panel de notificaciones** con badge rojo de no leídas, tipos diferenciados por color (riego/cosecha/alerta/info), marcar leídas individualmente o todas
 - **Formulario de cosecha desglosado** con selector de siembra activa desde /api/siembras, botones de calidad percibida y validación antes de enviar
-

Prompt #9 — Corrección de bug con verificación ampliada

Contexto:

Al abrir la pantalla de Cosecha la app se rompía con un error de JavaScript visible en la consola del navegador. Se reportó con screenshot y se aprovechó para solicitar revisión general del patrón.

Prompt utilizado:

[imagen de la consola del navegador]
"Uncaught TypeError: cosechas.reduce is not a function at Cosecha.jsx:25"

sale este error al abrir la pantalla de Cosecha. Además el botón de cerrar sesión no aparece en ninguna pantalla. Solucióvalo y verifica si existen otras incongruencias en el resto de las pantallas con el mismo patrón de acceso a datos del backend.

Por qué fue efectivo:

La efectividad viene de tres elementos combinados. Primero, la imagen del error mostró el mensaje exacto y el stack trace sin necesidad de transcribirlo. Segundo, agregar el segundo problema ("*botón de cerrar sesión*") en el mismo prompt evitó un intercambio adicional. Tercero — y más importante — la frase "*verifica si existen otras incongruencias con el mismo patrón*" transformó un prompt de corrección de un bug puntual en una auditoría general del código. Sin ese agregado, solo se habría corregido Cosecha.jsx.

Resultado obtenido:

- **Bug principal resuelto:** el backend devuelve { ok, data: { detalle: [] } } pero el código intentaba .reduce() directamente sobre el objeto — fix con `Array.isArray(resp?.data?.detalle) ? resp.data.detalle : []`
 - **Botón cerrar sesión** agregado en barra superior permanente visible en todas las pantallas, con nombre del vecino y rol
 - **Mismo patrón detectado y corregido** en Siembras.jsx e Inicio.jsx — bugs que habrían aparecido al navegar a esas pantallas
 - **Normalización unificada** del acceso a datos del backend en todos los componentes del proyecto
-

Patrones de Efectividad

El análisis de los 9 prompts revela técnicas consistentes que explican su efectividad:

Patrón	Cómo se aplicó
Contexto acumulativo	Los prompts más cortos funcionaron mejor cuando había más contexto previo. "ahora necesito el backend" generó 9 módulos porque Claude ya tenía el esquema completo de la BD
Imagen del error	Los 3 errores técnicos (ENUM, SSL, cosechas.reduce) se resolvieron en un único intercambio. La imagen mostró más información que cualquier descripción en texto
Restricciones como diseño	"sin necesitar cambiar el .env" forzó una solución robusta de detección automática de entorno en lugar de un parche de una línea
Verbos de acción específicos	"activar, agregar, arreglar, guarde, enviar" — cada verbo fue una instrucción independiente que impidió que algún requerimiento quedara sin implementar
Verificación ampliada	"verifica si existen otras incongruencias" transformó la corrección de 1 bug en la detección de 3 adicionales
Visual + contexto técnico	La imagen del prototipo Stitch + "ya tengo listo el backend" generó un frontend coherente y conectado sin especificaciones de diseño adicionales
Lenguaje de usuario	"el coordinador define un patrón y el sistema genera los turnos" — descripciones en lenguaje de negocio permitieron que Claude infiriera la arquitectura técnica correcta
Refactorización en el momento justo	"rehacerla desde cero con todo lo que llevamos" cuando el proyecto había madurado generó una BD v2.0 más completa que la suma de los cambios incrementales